

越权漏洞研究报告

越权漏洞（Broken Access Control）是指应用程序中权限控制设计或实现存在缺陷，导致攻击者可以访问**未经授权**的资源或功能。根据攻击者改变的对象与权限级别，常见的越权漏洞可分为**水平越权**、**垂直越权**和**交叉越权**^①。水平越权指**同一权限级别的不同用户互相访问对方资源**，如用户 A 通过修改请求参数查看用户 B 的数据^①；垂直越权指**低权限用户访问高权限用户资源**，如普通用户访问管理员功能^①；交叉越权则同时改变目标用户 ID 和权限类型，兼具水平和垂直越权特征^②。例如，某“三员系统”中，攻击者先以管理员 A 身份抓包、再替换为管理员 B 的会话 cookie 发送请求，如果仍能访问，则属于交叉越权^③。越权漏洞的根源在于服务器端对客户端请求**过度信任而忽略权限判定**^④^①，攻击后果可能包括敏感信息泄露、功能被非法操作等。

漏洞挖掘方法（针对PHP Web系统）

- **代码审计与静态分析**：在 PHP 源码中查找权限控制逻辑。重点关注控制流程中对 `$_GET` / `$_POST` 等客户端参数的使用，是否缺少对当前登录用户（`$_SESSION['user_id']` 或角色）的校验。如发现数据库查询直接使用用户传入的 ID、角色参数而未与会话信息比对，就可能存在越权风险。^④指出，开发者过分信任客户端数据而未做权限判定即是越权漏洞成因；审计时应验证这些操作是否正确使用了会话中的用户标识并授权。
- **黑盒渗透测试**：在无需源码的情况下，通过模拟用户操作和参数变异来发现漏洞。首先准备多个测试账号（普通用户、管理员等）。使用工具（如 Burp Suite、Postman 等）抓取正常操作的 HTTP 请求包，重点记录涉及用户标识或资源标识（如用户 ID、订单号、文件名）的参数。然后对这些参数进行测试：如**枚举**、**遍历**或**修改**（例如尝试将用户 A 的参数改为用户 B），观察响应是否未作拒绝。对于 API 接口，还可使用脚本（如 `curl`、Python requests）模拟请求，例如：

```
curl -X GET "http://target/app.php?action=viewProfile&userid=1" -b "PHPSESSID=abc123"
curl -X GET "http://target/app.php?action=viewProfile&userid=2" -b "PHPSESSID=abc123"
```

如果第二条请求仍然返回了用户 2 的信息，则说明存在水平越权漏洞。测试中可使用漏洞扫描器或代理插件辅助，但需要人工验证结果，特别是在影响数据范围广泛时要**多账号对比**^⑤。

- **动态爬虫与暴力试探**：利用脚本（如 Python + `requests`）自动枚举 URL 和参数。例如爆破常见的管理后台路径、接口地址；对可控 ID 进行遍历，判断访问结果差异；尝试在 AJAX 或 REST API 中插入不同用户的标识；注意处理 JSON 响应中的字段。还可结合 Spider 或 Fuzzer（如 OWASP ZAP、Burp Intruder）模拟大规模测试，以发现隐藏的越权入口。
- **多环境对比**：对于同一系统的不同部署环境（如测试环境、生产环境），常常存在配置差异。可尝试在测试版无感知情况下修改请求中 `Host` 或其他头，将流量导向不同环境，看是否绕过授权。

使用 Burp Suite 进行越权漏洞挖掘

1. **配置代理**：启动 Burp Suite，设置浏览器代理为 Burp。用 Burp 内置的浏览器或配置系统代理后，在浏览器中以普通用户身份登录目标 PHP 应用，并进行正常操作（如查看个人信息）。在 Proxy 的 **HTTP history** 中找到关键请求（例如含有用户 ID、订单号等参数的请求）。
2. **参数修改与重复发送**：将目标请求右键发送到 **Repeater**。在 Repeater 中修改请求中的敏感参数，例如将 `userid=1` 改为 `userid=2`、或将 `contactId=xxx` 改为其他值，然后点击 **Go** 发送。检查响应内容：若返回了不属于当前用户的数据（如用户 B 的信息），则存在水平越权漏洞。类似地，对对象 ID、文件 ID 等均可尝试修改。

3. **权限提升测试**：寻找只有高权限用户能访问的功能（如管理员页面、敏感配置接口）。截获该请求，将自己的登录 session 或 token 替换成普通用户的。在 **Cookie Jar** 或 **Repeater** 中手动修改 Cookie（或 Authorization 头）字段。例如，有的系统会在 Cookie 中直接存储角色标识，可尝试将 `user=guest` 改为 `user=admin` 再刷新页面⁶。如果普通用户通过修改后依然能访问到管理员功能，就发现了垂直越权漏洞。
4. **自动化辅助**：可以使用 Burp 的 **Intruder** 或 **Autorize** 插件批量测试。Authorize 插件能在后台拦截请求、快速替换不同用户令牌并检查响应状态，从而半自动发现越权⁷。例如，用 Authorize 按钮启动拦截，填写一个无权限的用户 token，再点击目标功能；插件会尝试用不同用户的 token 并高亮有越权风险的请求⁷。发现疑似漏洞后，同样点击 **Send to Repeater**，进一步调试确认。
5. **案例操作示例**：假设用户 A 登录后，抓取了查看订单的请求：

```
GET /order/view?id=1001 HTTP/1.1
Cookie: PHPSESSID=abcd1234; user_id=5
```

在 Repeater 中，将 `id=1001` 改为另一用户的订单号 `id=1002`，并重发请求。若返回了订单 1002 的信息，则 A 对 B 的订单实现了水平越权。类似地，可在管理员页面请求中直接粘贴普通用户 cookie，若仍能加载管理界面，则存在垂直越权。

漏洞挖掘技巧

- **逻辑盲点利用**：设计者对业务流程的假设往往是漏洞点。需要梳理应用的业务流程与角色功能边界，找出可能“漏检”的节点。例如，某些接口只在前端做了角色判断，而后端忽略验证；或在一个功能内判断了权限，但其子功能未重复验证，就可造成越权。可重点检查新增、修改后再查询的流程：如果新增接口未绑定用户 ID，导致之后的查询可以任意查看记录，就会发生越权。在测试时，通过跟踪接口链（Chain of custody）发现数据关联处的漏洞。
- **接口访问控制缺陷**：很多 Web 系统使用多层接口，有时权限检查不一致。可以测试同一功能的多种访问路径：如页面访问限制和 API 接口限制可能不同；亦可测试不同请求方式（GET/POST/JSON）下的权限控制是否一致。有时可以利用“防护差异”：例如 A 账户可以发起操作，但 B 账户无法完成同类操作，或在某些接口只有作用于高权限的参数验证。对 RESTful API，可检查 URL 中是否存在敏感 ID（如 `/admin/user/delete/{id}`），尝试将 `{id}` 改为其他用户并发送。
- **Cookie/Session 机制绕过**：如前文所示，一些系统在 Cookie 或会话中存储可控的权限标志或用户标识⁶³。可以使用浏览器插件（Cookie Editor）或 Burp 来修改这些值。如将普通用户的 `role=guest` 修改为 `role=admin`，或利用管理员会话复制到普通用户的请求包中测试权限。若应用仅依赖 Cookie 里的简单变量进行鉴权，就可被轻易绕过⁶³。此外，还可检测 SessionID 是否可预测（例如碰撞攻击）或是否在多个账号中共用，以便通过猜测或复用会话获得他人权限。
- **标识枚举与高权限动作**：有时可以通过试探高权限参数值来发现越权。例如 API 返回的 JSON 中可能包含敏感 ID，可尝试操纵这些 ID 参数；也可以测试是否能替换某些内部引用（如文件下载的文件名、资源路径），以访问本应无权下载的文件。合法用户下载文件时，将 URL 里的文件名替换为他人的文件 ID，如果能成功下载，即存在越权（如[14 + L125-L133]所示，通过遍历 `fileid` 下载到其他用户文件）。
- **注意未授权访问与结束会话**：在测试越权前，先确认系统是否存在未授权访问（Unauthenticated Access）或“记住我”登录失败的情况。如果登录后用户退出但会话未失效，攻击者可能仍能使用过期的 Cookie 继续访问。在进行越权测试前，确保这些基础问题已排除，否则可能误判。

真实案例分析

1. **HackerOne 平台漏洞（IDOR）**：2024 年，白帽黑客在 Bug Bounty 平台 HackerOne 上发现一个严重越权漏洞⁵。该漏洞出现在报告附件功能中：平台允许用户在漏洞报告中上传附件，但不检查附件所属用户。攻击者在编辑报告摘要时，通过**修改请求中的附件 ID**，就可以下载其他用户账户下的敏感附件⁵⁸。这属于不安全直接对象引用（IDOR）漏洞，严重危及用户数据机密性：攻击者无需权限即可

查看他人提交的漏洞 PoC 图片、视频等⁵。HackerOne 于漏洞公开后为该漏洞支付了 1.5 万美元的奖励，并修复了验证逻辑，确保附件 ID 必须与当前用户匹配才能访问（即在后台比对文件所有权）。此案例凸显了即使是安全产品自身，也可能忽略细粒度的访问控制。

2. **WordPress Paid Memberships Pro 插件漏洞（CVE-2024-37277）**：此插件为 WordPress 提供付费会员功能，版本 3.0.4 及之前存在 IDOR 漏洞⁹。攻击者可通过控制用户可控的键（key）参数绕过授权，访问未受访问控制限制的功能⁹。具体来说，恶意用户构造请求时操控该 key 值，绕过了后台的访问控制，进而可以查看或操作本应受限的数据⁹。利用该漏洞，攻击者可能读取其他会员的私人信息或升级自己的权限。厂商已在 2024 年 6 月发布修复，要求用户升级至版本 3.0.5 或更高¹⁰。修复方法是在服务端对该参数进行校验，不信任用户输入，并且严格在数据查询中加入当前用户 ID 的过滤条件，以防止未授权访问。

3. **WordPress Miraculous Core 插件漏洞（CVE-2025-58627）**：该插件用于管理网站的幻灯片效果，低于 2.0.9 版本存在授权绕过漏洞¹¹。原理是插件对访问控制配置不当：攻击者可构造特定请求，通过控制某个键（用户可控参数）来绕过授权验证¹¹。利用该漏洞，攻击者可以访问本不应开放给自己的管理功能或数据，造成敏感信息泄露和权限滥用¹²。影响范围包括所有未升级的 Miraculous Core 插件用户。该漏洞已公开披露，开发者在 2.0.9 版本中修复了访问控制逻辑（仅允许经过认证的用户操作其有权限的资源），并建议用户及时更新插件。

以上案例说明，越权漏洞往往产生于访问控制判断的缺失或错误配置，只要绕过验证检查即可导致严重影响。及时修复通常包括在后台严格校验用户身份和权限，例如在查询语句中关联 user_id、加入权限开关或角色判断等。对于开源组件，还应关注安全更新，使用厂商发布的补丁或新版本。

漏洞整改措施（针对PHP系统）

- **服务端权限验证**：绝不能依赖客户端参数来决定权限。应在 PHP 代码中从会话（\$_SESSION）或服务端存储中获取当前用户信息，然后严格校验当前操作是否属于该用户或其角色权限范围¹³。例如，对于用户只能查看自己的情况，可这样编写：

```
<?php
session_start();
$currentUser = $_SESSION['user_id'];
if (!$currentUser) {
    header('HTTP/1.1 401 Unauthorized');
    exit('请先登录');
}
$requestedId = intval($_GET['id']);
if ($requestedId !== $currentUser) {
    header('HTTP/1.1 403 Forbidden');
    exit('无权访问');
}
// 仅查询属于当前用户的数据
$stmt = $pdo->prepare('SELECT * FROM users WHERE id = ?');
$stmt->execute([$currentUser]);
```

以上代码示例中，后端没有信任 \$_GET['id']，而是强制只能读取与当前登录用户一致的记录。正如文献所建议的：“涉及用户隐私的操作时，从 Session 中取出用户标识进行操作，不要轻信用户的输入”¹³。

- **角色权限校验**：对于需要区分普通用户与管理员的功能，应在 PHP 后端为每个接口或页面增加角色验证。例如：

```
<?php
session_start();
if ($_SESSION['role'] !== 'admin') {
    header('HTTP/1.1 403 Forbidden');
    exit('仅限管理员访问');
}
// 管理员功能代码
```

在执行高权限操作前，检查 `$_SESSION['role']` 或类似标识。即使前端隐藏了管理入口，也要在后端再次进行判断。可以使用常见的 RBAC 框架（如 Laravel 的授权门面、Symfony 的 voter）或者自定义函数来统一管理权限，比如在通用函数中检查用户是否有某项操作的许可。

- **接口细粒度授权**：对于 RESTful API，应该在每个查询或修改操作时都关联用户标识。比如：

```
<?php
session_start();
$currentUser = $_SESSION['user_id'];
$orderId = intval($_GET['order_id']);
$stmt = $pdo->prepare('SELECT * FROM orders WHERE id = ? AND user_id = ?');
$stmt->execute([$orderId, $currentUser]);
```

这样只有当 `orders.user_id == $currentUser` 时才会返回数据，否则因条件不匹配而返回空，从而防止普通用户查看他人订单。切勿在 SQL 语句中直接使用来自用户输入的 ID 作为查询条件而不加筛选。

- **不在 Cookie/客户端存储敏感权限**：避免在 Cookie、Hidden 表单字段或 URL 中存放可被篡改的权限标识。若必须将信息传递给前端，应加密签名或在服务端检验。例如，若用 JWT 等方式，服务器在解码时要验证签名并检查 payload 中的用户 ID。Cookie 中不应直接保存 `role=admin` 之类的字段，否则攻击者易于仿造⁶。
- **最小授权原则与会话管理**：每个用户只授予其最低限度的权限，服务端应避免使用共用简单角色标志来判断访问权限。如果发现某些高权限操作只被客户端（如前端 JS）控制，应改为服务器端控制验证。并且每次高敏感度操作前都应验证用户身份和权限，防止会话劫持或权限提升攻击。还要注意用户登出时销毁会话，避免未清理登录状态导致的越权。

通过以上措施，即可从编码和设计层面降低越权漏洞发生的风险。总之，任何涉及用户隐私或敏感功能的需求，都应严格基于服务器端的身份验证和授权逻辑，而不是盲目信任用户提供的任何参数¹³。

¹ ² ¹³ 渗透测试逻辑漏洞原理与验证(4)——权限控制问题-腾讯云开发者社区-腾讯云
<https://cloud.tencent.com/developer/article/2476202>

³ “三员系统”中常见的越权问题 - FreeBuf网络安全行业门户
<https://www.freebuf.com/articles/web/235153.html>

⁴ 怎么解决php中的越权访问,WEB安全系列之《如何挖掘越权漏洞》-CSDN博客
https://blog.csdn.net/weixin_34170028/article/details/115809637

⁵ ⁸ 最新H1越权漏洞披露，获得15000美元奖励 - Guge's Blog
<https://gugesay.com/archives/2647>

6 逻辑漏洞-其四（越权漏洞）_平行越权-CSDN博客

https://blog.csdn.net/qq_64177395/article/details/142694697

7 使用2023版BurpSuite半自动化Authorize进行垂直越权测试【图文教程】_burpsuite authorize-CSDN博客

https://blog.csdn.net/weixin_45807466/article/details/138137807

9 10 WordPress Paid Memberships Pro plugin <= 3.0.4 - Insecure Direct Object References (IDOR)

vulnerability 漏洞信息(CVE-2024-37277) - by 漏洞平台

<https://cve.imfht.com/detail/CVE-2024-37277>

11 12 WordPress Miraculous Core Plugin plugin < 2.0.9 - Insecure Direct Object References (IDOR)

vulnerability 漏洞信息(CVE-2025-58627) - by 漏洞平台

<https://cve.imfht.com/detail/CVE-2025-58627>